

剩下的模式



不是每个模式都广受欢迎。距离《Design Patterns: Elements of Reusable Object-Oriented Software》首次出版，已经过去了超过25年，很多事情已经变了。开发人员大量应用这些模式。我们在这个附录中所概括的模式，都是成熟、典型、正式的GoF模式，只不过可能不像我们之前所探索的模式那么经常被使用。但是这些模式也自有可取之处，如果你的情况刚好适用，你应该毫不犹豫地用上它们。这个附录的目标，是从概述的角度告诉你这些模式是关于什么的。

桥接 (Bridge)

使用桥接模式不只改变你的实现，也改变你的抽象。

场景

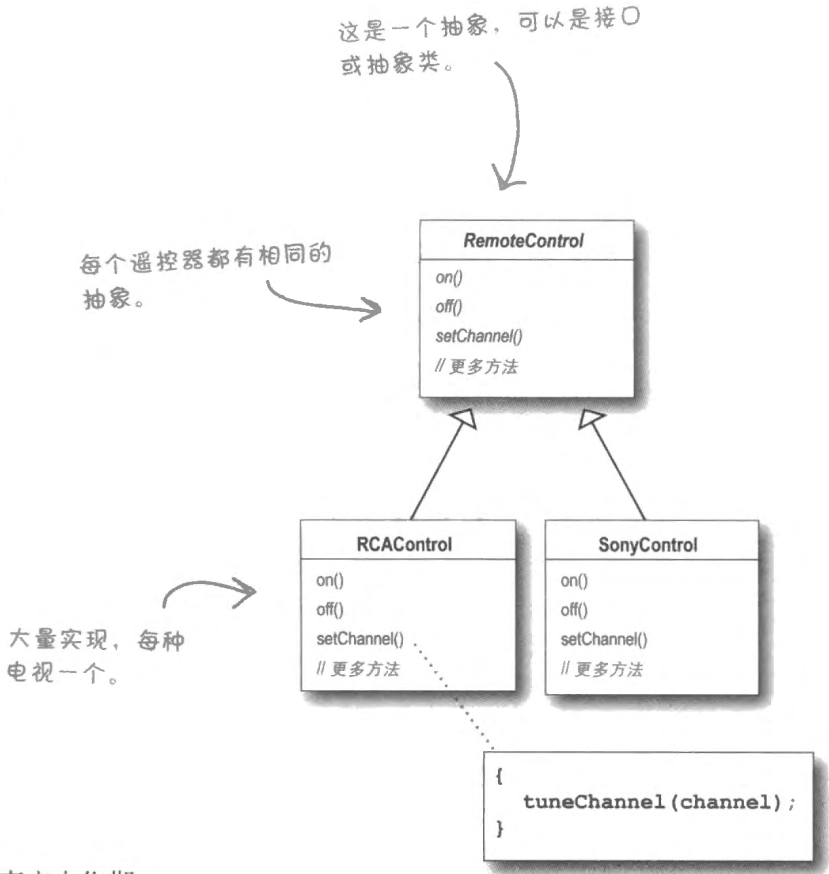
想象你正在为一个新的符合人体工学、用户友好的电视遥控器编写代码。你已经知道要使用好的面向对象技能，让所有遥控器基于相同的抽象，然后得出很多实现，每个型号的电视一个实现。

你的困境

你知道遥控器的用户界面不会第一次就做对。事实上你期望随着遥控器的可用性数据的收集，遥控器会被多次改良。

因此你的困境是：遥控器会变化，而电视机也会变化。你已经把用户界面抽象出来，以便根据顾客所拥有的众多电视机改变实现。你也需要改变抽象，因为随着时间的推移，遥控器也会基于用户反馈改进。

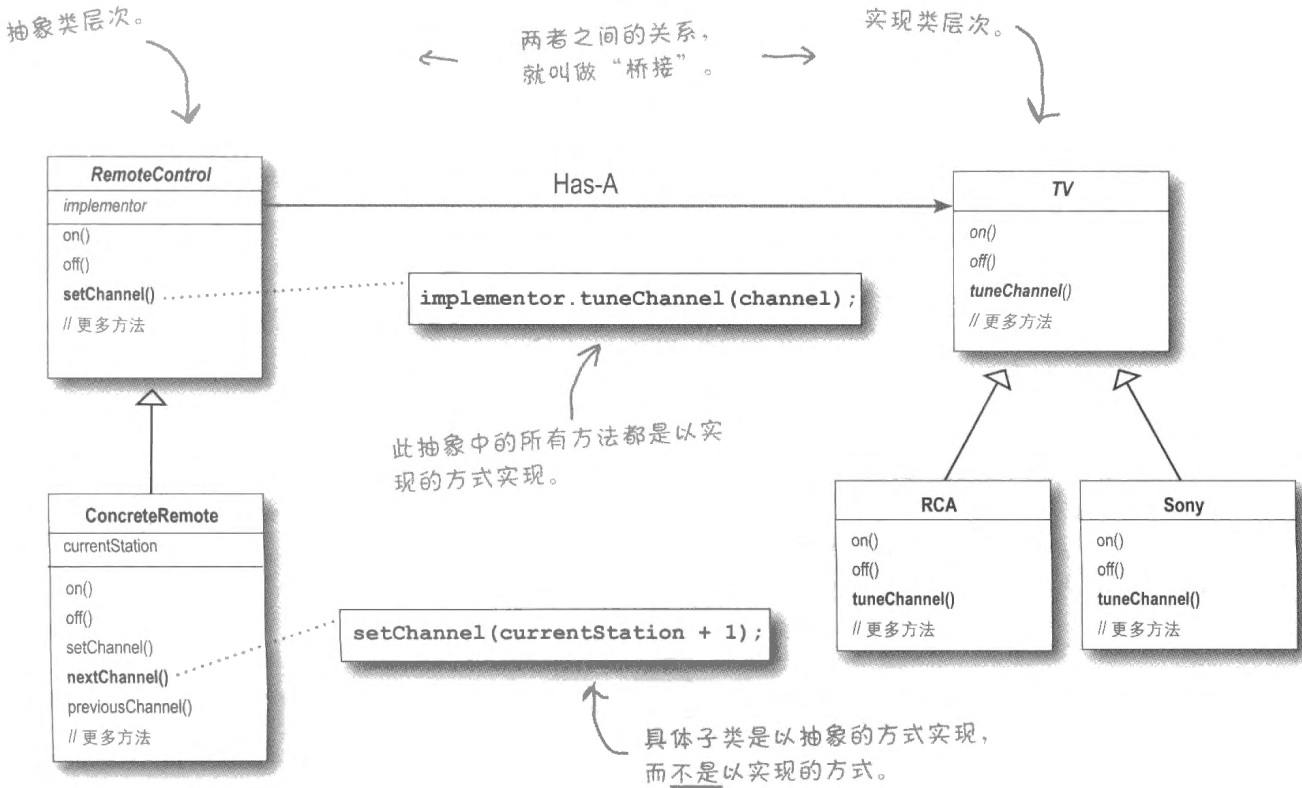
因此，你要如何创建一个面向对象设计，能让你改变实现和抽象？



使用这个设计，我们可以只改变电视的实现，而用户界面不变。

为什么使用桥接模式？

通过把实现和抽象放在两个分离的类层次中，桥接模式可以让你独立改变它们。



现在你有了两个层次结构，其中一个是遥控器，另一个分开的是平台特定的电视机实现。桥接能让你独立地改变这两个层次的每一端。

桥接的优点

- 解耦实现，不再永久绑定到接口。
- 抽象和实现可以独立扩展。
- 针对“具体的抽象类”的变化，不会影响客户。

桥接的用途和缺点

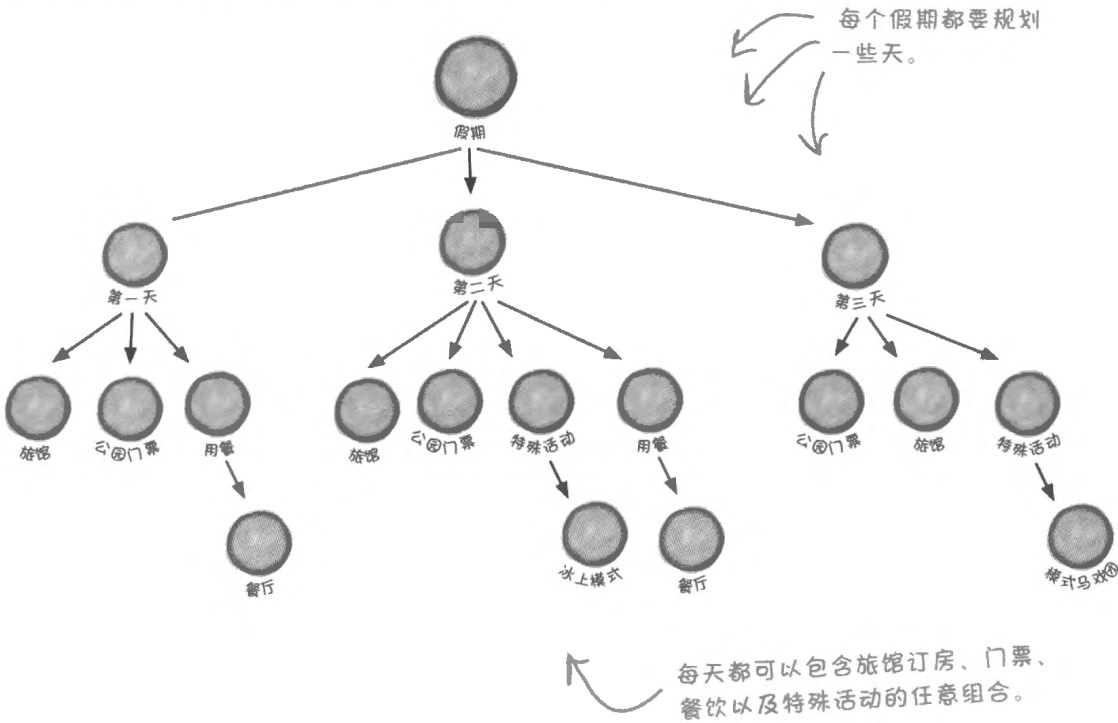
- 在跨多个平台的图形和窗体系统中很有用。
- 当你需要以不同的方式改变接口和实现时，桥接很好用。
- 缺点是增加了复杂度。

生成器 (Builder)

使用生成器模式来封装一个产品的构造过程，并允许按步骤构造。

场景

你刚被要求为模式乐园构造一套度假规划。模式乐园是在对象村外围的一座新的主题公园。公园的客人可以选择旅馆以及各种门票、餐厅订位、甚至预约特殊活动。想要创建一套度假规划，你需要创建像下面这样的结构：



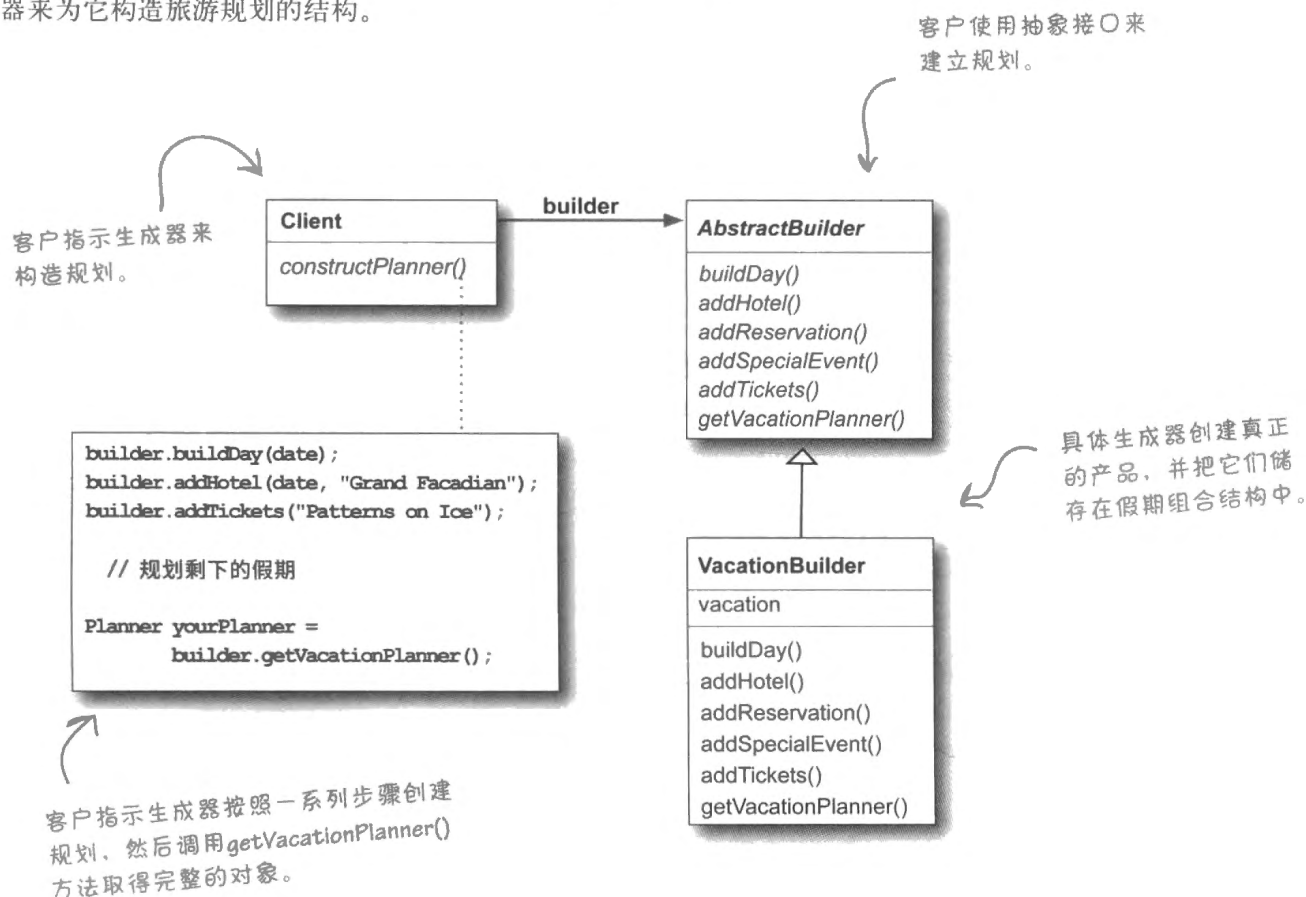
你需要一个有弹性的设计

每个客人的度假规划的天数、活动类型可能都不太一样。例如，当地居民可能不需要旅馆，但是想要用餐并预约特殊活动。另一位客人可能是坐飞机到对象村，因此需要旅馆、订餐和门票。

因此，你需要一个有弹性的数据结构，表达客人的计划以及所有的变体，还需要遵循一个潜在的复杂步骤序列来创建计划。你怎样才能提供一种方式，创建这个复杂的结构，而不会和创建它的步骤混在一起？

为什么使用生成器模式？

还记得迭代器吗？我们将迭代封装进一个分离的对象，并对客户隐藏集合的内部表示。这里也是相同的思路：我们把旅游规划的创建封装进一个对象中（我们称为生成器），然后让客户请求生成器来为它构造旅游规划的结构。



生成器的优点

- 封装复杂对象的构造方式。
- 允许对象用多个步骤构造，并且可以改变过程（和一步到位的工厂不同）。
- 向客户隐藏产品的内部表现。
- 产品的实现可以被替换，因为客户只看到抽象的接口。

生成器的用途和缺点

- 经常被用来建造组合结构。
- 和使用工厂相比，构造对象需要更多客户的领域知识。

责任链 (Chain of Responsibility)

当你要把一个处理请求的机会给予多于一个的对象时，使用责任链模式。

场景

自从发布Java版本的糖果机之后，万能糖果公司收到的电子邮件数量，超出了他们所能处理的范围。他们自己分析，收到的电子邮件有四类：喜欢新的1-in-10游戏的顾客粉丝来信；父母寄来的信，抱怨他们的孩子沉溺于这个游戏；在新的地方摆设糖果机的请求；相当数量的垃圾邮件。

所有粉丝邮件应该直接送到CEO手上，所有抱怨应该去往法务部门，而所有的新机器请求应该去往业务拓展部门，垃圾邮件应该被删除。

你的任务

万能糖果公司已经写了一些AI检测器，可以分辨邮件属于上述哪一类，但是它们需要创建一个设计，使用这个检测器来处理收到的邮件。

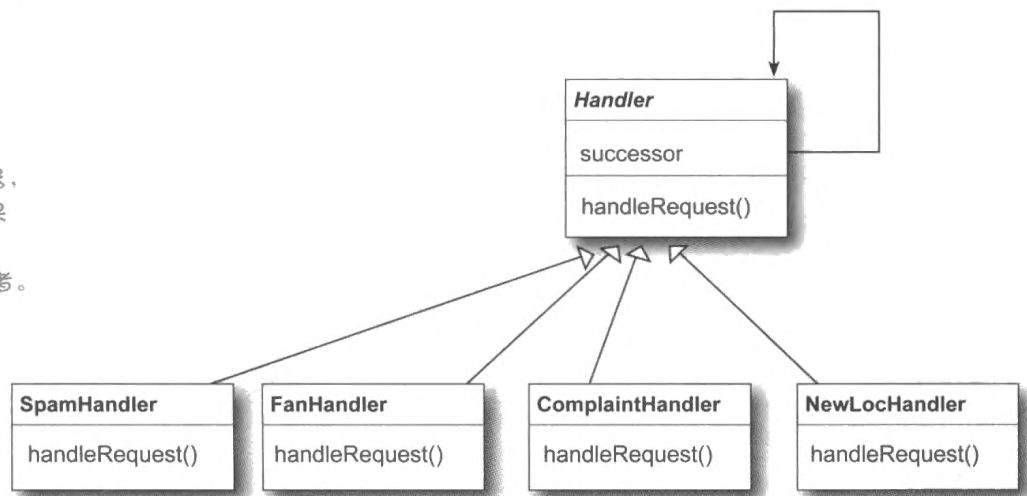
你一定要帮我们处理这洪水般的电子邮件，自从Java糖果机推出后，邮件数量大增。



如何使用责任链模式

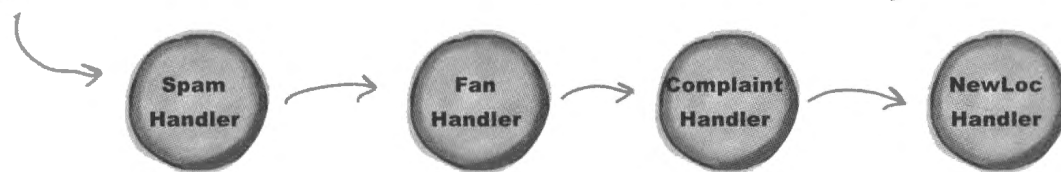
通过责任链模式，你可以创建一个对象链来检查请求。每个对象依次检查请求，处理它或者将它传给链中的下一个对象。

链中的每个对象扮演处理器，并且有一个后继对象。如果可以处理请求，就处理，否则把请求转发给其后继者。



收到电子邮件时，它会传给第一个处理器：SpamHandler。
如果SpamHandler无法处理该请求，就将它传给FanHandler。
依此类推……

每个电子邮件被传给第一个处理器。



如果电子邮件掉落到链的末尾，就表示它没有被处理，不过你总是可以实现一个扫尾处理器。

责任链的优点

- 解耦请求的发送者和接收者。
- 简化你的对象，因为它不需要知道链的结构以及保持对其成员的直接引用。
- 允许你通过改变链的成员或次序，动态地添加或移除责任。

责任链的用途和缺点

- 经常用在窗体系统中，处理鼠标点击和键盘事件。
- 并不保证请求一定会被执行；如果没有对象处理它，可能会掉到链的末尾（这可以是优点或缺点）。
- 运行时可能会难以观察和调试。

蝇量 (Flyweight)

当某个类的一个实例可以用于提供许多虚拟实例时，使用蝇量模式。

场景

在你的新景观设计应用中，你要添加一些树作为对象；在你的应用中，树不做太多事情；它们有一个XY位置，可以根据树的年龄动态地绘制自己。问题是，用户可能要在他们的家庭景观设计中，放非常非常多的树，看起来像这样：

Tree
xCoord
yCoord
age
<pre>display() { // 使用xx坐标 // 以及复杂的 // 树龄相关的计算 }</pre>

你的大客户的困境

你有一个已经努力推销了好几个月的关键客户，他们打算购买1000套你的应用，他们用你的软件为大型规划社区做景观设计。在使用你的软件一个星期之后，客户抱怨，在创建了许多树之后，应用开始变得迟缓……

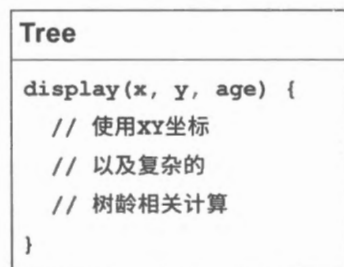
为什么使用蝇量模式?

你可以重新设计你的系统，不用大量的对象，只有一个树实例以及一个维护你所有树的状态的客户对象。这就是蝇量！

所有虚拟树对象的所有状态，储存在这个二维数组中。



一个，单独一个，没有状态的树对象。



蝇量的优点

- 减少运行时对象实例的数目，节省内存。
- 把许多“虚拟”对象的状态集中放进一个地方。

蝇量的用途和缺点

- 当一个类有许多实例，而这些实例能够用一致的方法控制时，用蝇量。
- 蝇量模式的一个缺点在于，一旦你实现了它，那么类的单个逻辑实例，将无法拥有和其他实例不同的独立行为。

解释器 (Interpreter)

使用解释器模式为语言建造解释器。

场景

还记得鸭子模拟器吗？你觉得它也可以是儿童学习编程很好的教育工具。使用这个模拟器，每个孩子都可以用简单的语言控制一只鸭子。该语言的简单例子如下：

```
right;
while (daylight) fly;
quack;
```

让鸭子右转。
飞一整天……
……然后呱呱叫

现在，回想很久以前你在入门编程课上学习的如何创建语法的知识，把语法写出来：

```
expression ::= <command> | <sequence> | <repetition>
sequence  ::= <expression> ';' <expression>
command  ::= right | quack | fly
repetition ::= while '(' <variable> ')' <expression>
variable  ::= [A-Z,a-z]+
```

程序是一个表达式，包含一个命令序列以及重复（“while”语句）。

一个序列，指的是一个用分号隔开的表达式的集合。

我们三个命令：向右、呱呱叫和飞行。

while语句由一个条件变量和一个表达式组成。

现在怎么办？

你有了一个语法，现在所需要的是一个表达并解释语法中句子的方式，以便可以看到他们在模拟鸭子上的编程效果。

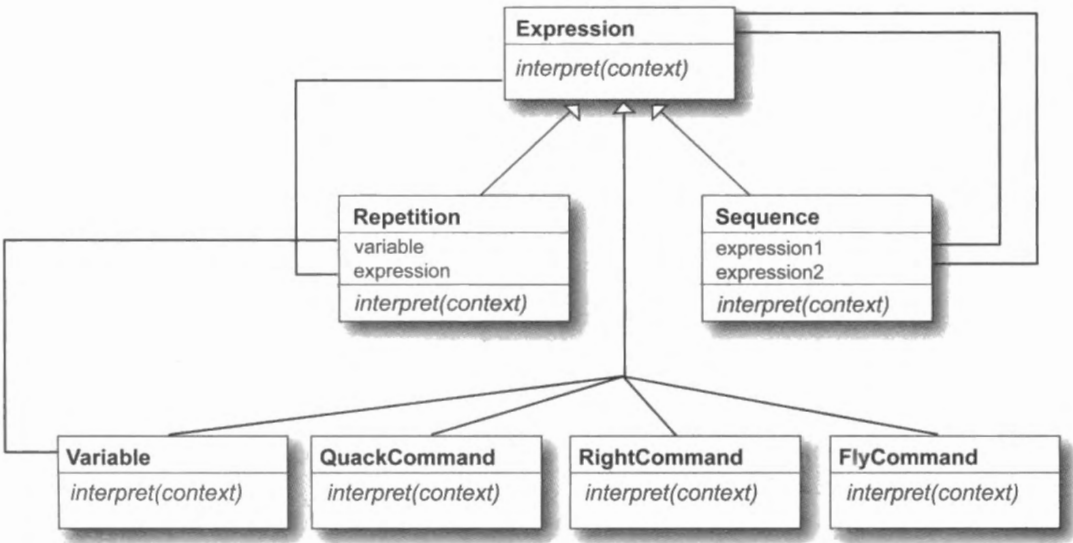


解释器模式需要一些形式语法的知识。

如果你从未学习过形式语法，继续阅读完该模式，你还是可以理解要点的。

如何实现解释器

当你需要实现一门简单的语言时，解释器模式为语言的语法以及解释句子的解释器定义了一个基于类的表达。为了表达语言，你用一个类来表达语言中的每条规则。这是一个将鸭子语言转化成类的例子。注意，类直接映射到语法。



要解释语言，就调用每个表达式类型上的interpret()方法。这个方法需要传入一个context（上下文），它包含我们正在解析的程序的输入流，并匹配输入和求值。

解释器的优点

- 将每一个语法规则表达成一个类，使得语言容易实现。
- 因为语法由类表达，你可以轻易地改变或扩展该语言。
- 通过在类结构中添加方法，可以添加解释之外的新行为，例如打印格式的美化和更复杂的程序验证。

解释器的用途和缺点

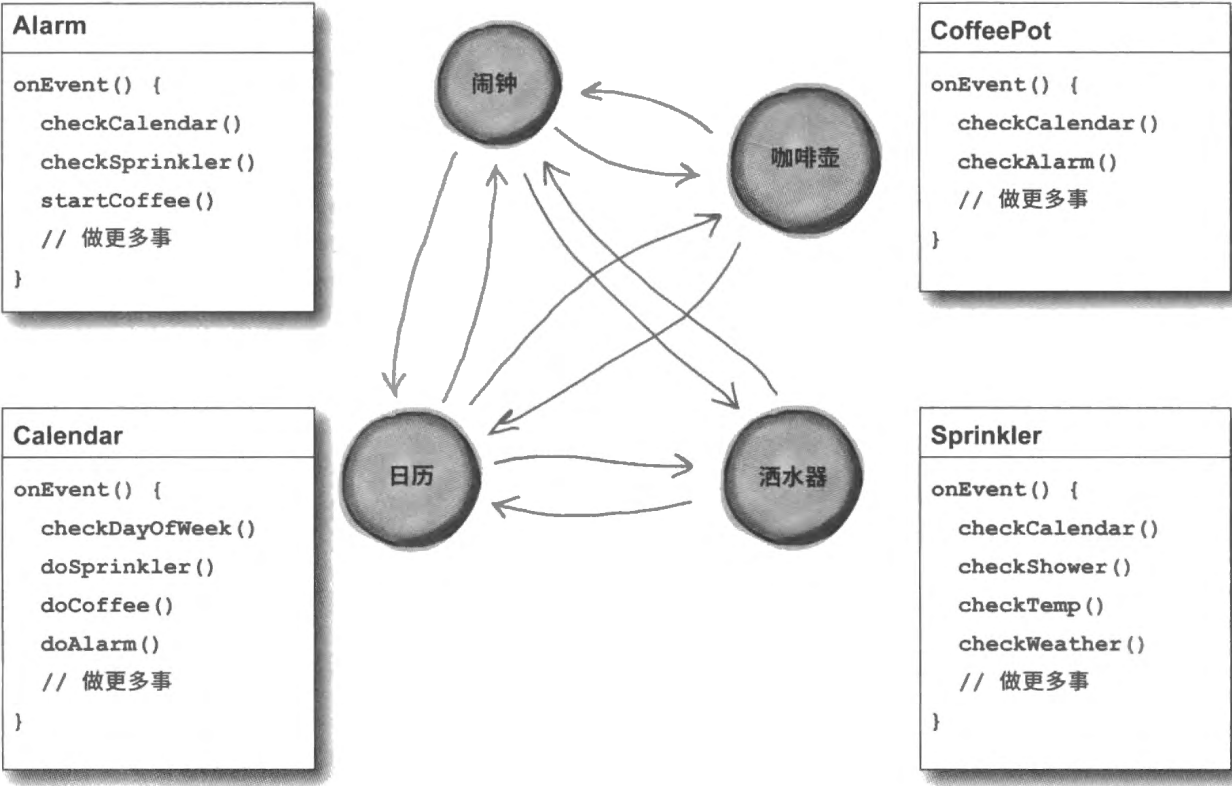
- 当你需要实现一门简单的语言时，使用解释器。
- 当你有一个简单的语法，而且简单比效率重要时，适合用解释器。
- 用于脚本和编程语言。
- 当语法规则的数目很大时，这个模式可能变得笨重。在这些情况下，一个解析器/编译器的生成器可能更适合。

中介者 (Mediator)

使用中介者模式来集中相关对象之间复杂的沟通和控制方式。

场景

感谢未来屋公司的这帮好人，Bob有了一个自动化的屋子。所有电器都设计成让他的生活更便利。当Bob按停“小睡一会”的按钮，他的闹钟就告诉他咖啡壶开始煮咖啡了。尽管生活对Bob来说非常美好，他（以及其他顾客）还是不断要求许多新特性：周末不要咖啡……在洗澡时间的前15分钟将洒水器关闭……把丢垃圾日的闹钟设得早一些……



未来屋公司的困境

要跟踪哪条规则驻留在哪个对象中，以及各种对象之间如何彼此相关，确实很难。

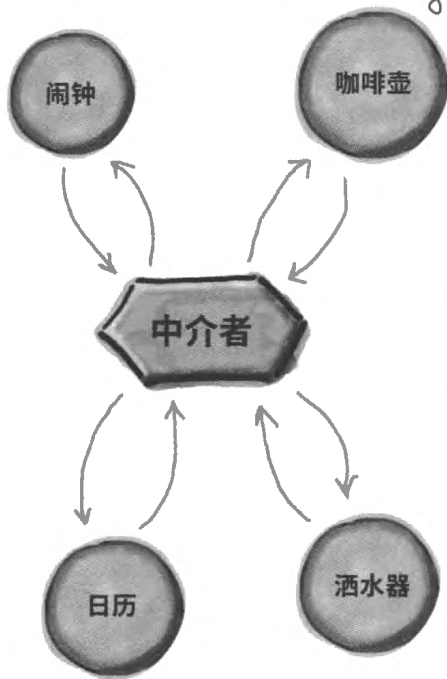
中介者在行动……

把一个中介者加到系统中，所有电器对象都可以大大简化：

- 当它们的状态变化时，告诉中介者。
- 它们响应来自中介者的请求。

在添加中介者之前，所有电器对象都需要知道彼此……也就是说，对象之间是紧耦合的。中介者到位后，电器对象彼此之间彻底解耦。

中介者包含整个系统的控制逻辑。当一个已有电器需要一个新规则或者一个新的电器被加入系统中时，所有必需的逻辑会被加到中介者内。



Mediator

```
if (alarmEvent) {
    checkCalendar()
    checkShower()
    checkTemp()
}
if (weekend) {
    checkWeather()
    // 做更多事
}
if (trashDay) {
    resetAlarm()
    // 做更多事
}
```

中介者的优点

- 通过将中介者支持的对象从系统解耦，增加对象的复用性。
- 通过将控制逻辑集中，简化了系统的维护。
- 简化以及减少系统中对象之间发送的消息的变化。

中介者的用途和缺点

- 中介者常用于协调相关的GUI组件。
- 中介者模式的一个缺点是，如果设计不当，中介者对象本身会变得过度复杂。

备忘录 (Memento)

当你需要让对象返回之前的某个状态时，例如，你的用户请求“撤销”，使用备忘录模式。

场景

你的交互式角色扮演游戏取得了巨大的成功，很多人沉迷于其中，大家都想到达传说中的“13关”。当用户进入更有挑战性的游戏关卡时，遭遇游戏结束情境的概率增加了。对于那些花了很多天进展到高级关卡的粉丝来说，当他们的角色被干掉时，简直是气炸了，他们不得不从头再来。他们含泪要求一个“保存进度”的命令，好让玩家在他们的角色被不公正地消灭时，能够储存游戏进度，或者至少挽回大部分的努力。这个“保存进度”功能需要设计成返回一个复活的角色到最后一个成功完成的关卡。



小心，保存游戏状态相当复杂，我可不想让别人访问和搞坏我的代码。

备忘录在工作

备忘录有两个目标：

- 保存系统关键对象的重要状态。
- 维护关键对象的封装。

牢记单一责任原则，保持正在保存的状态和关键对象分离，是个好主意。这个持有状态的、分离的对象，称为备忘录对象。

这不是一个很精美的实现，不过注意，客户没有访问备忘录数据。

```
Client

// 到达新关卡时
Object saved =
    (Object) mgo.getCurrentState();

// 需要回档时
mgo.restoreState(saved);
```



备忘录的优点

- 保持被保存的状态处于关键对象外面，有助于维护内聚。
- 保持关键对象的数据封装。
- 提供容易实现的恢复能力。

备忘录的用途和缺点

- 备忘录用于保存状态。
- 使用备忘录的一个缺点是，保存和恢复状态可能相当耗时。
- 在Java系统中，考虑使用序列化（Serialization）来保存系统的状态。

原型 (Prototype)

当创建给定类的实例昂贵或复杂时，使用原型模式。

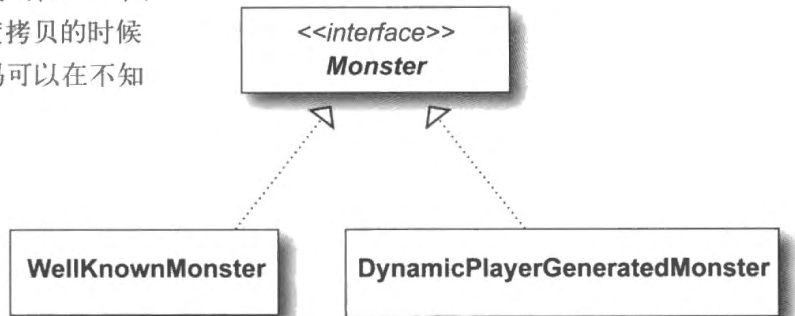
场景

你的交互角色扮演游戏，对于怪兽的胃口简直是贪得无厌。当你的英雄在动态创建的景物间游走时，他们要遭遇一波又一波的敌人并将其消灭。你希望怪兽的特征能够随着景物的变化而演变。毕竟，如果让鸟一般的怪兽，跟随你的角色进入海底世界，实在是没有道理。最后，你希望允许高级玩家创建他们自己的定制怪兽。



原型来拯救你

原型模式能让你通过复制已有实例来制作新实例（在Java中，这通常意味着使用clone()方法，或者在需要深度拷贝的时候反序列化）。这个模式的重点在于，客户的代码可以在不知道实例化哪个特定类的情况下，制作新实例。



```

MonsterMaker
makeRandomMonster() {
    Monster m =
        MonsterRegistry.getMonster();
}
  
```

客户需要一个适合当前情况的新怪兽（客户不知道得到哪一种怪兽）。

```

MonsterRegistry
Monster getMonster() {
    // 找到正确的怪兽
    return correctMonster.clone();
}
  
```

这个注册表找到合适的怪兽，做一个克隆，返回该克隆。

原型的优点

- 向客户隐藏制作新实例的复杂性。
- 提供一种选择，让客户生成类型未知的对象。
- 在某些环境下，复制对象比创建新对象更有效。

原型的用途和缺点

- 在一个复杂的类层次中，当系统必须创建许多类型的新对象时，应该考虑原型。
- 使用原型的缺点是，对象的复制有时候相当复杂。

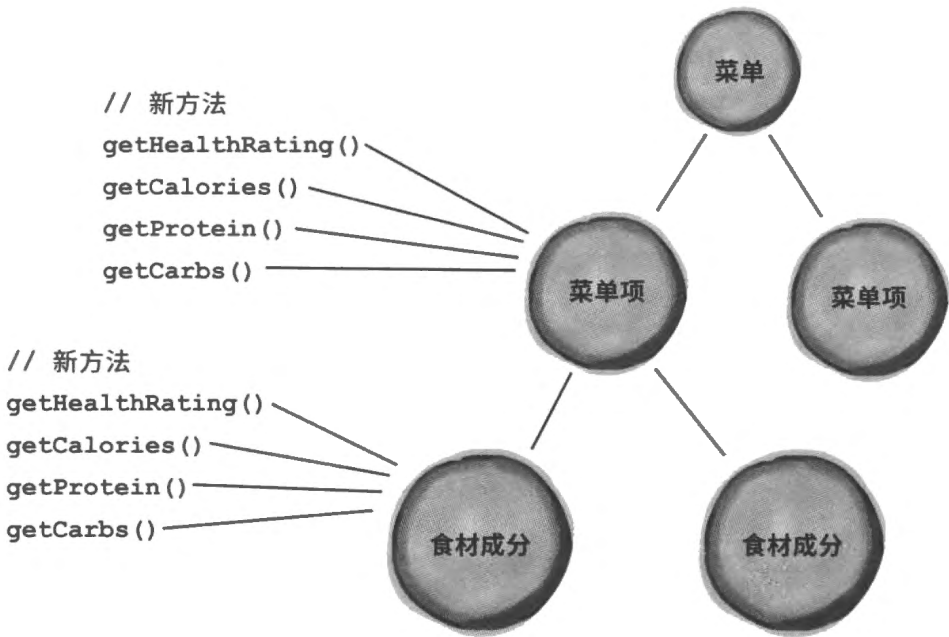
访问者 (Visitor)

当你想要为一个对象组合增加能力，且封装不重要时，使用访问者模式。

场景

对象村餐厅以及对象村煎饼屋的常客，近来变得非常有养生意识。在订餐之前，他们会询问营养信息。因为两家从业者都非常乐意为顾客定制，有些顾客甚至询问每种食材的营养信息。

Lou提出的解决方案：

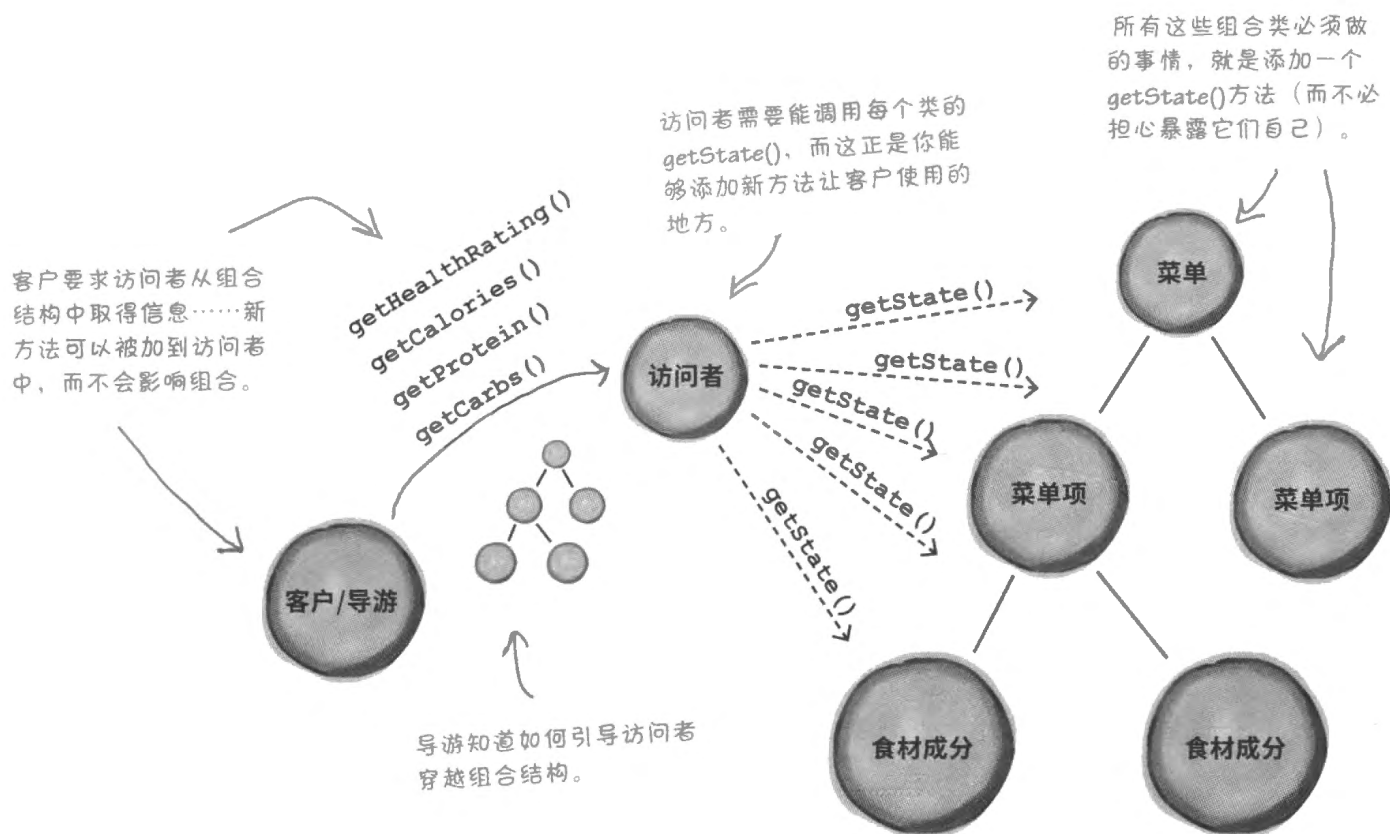


Mel的顾虑……

“好家伙，看样子我们打开了潘多拉的盒子。天晓得我们接下来得加入什么新方法，每一次我们添加新方法，不得不在两个地方做这件事。还有，万一我们要加强基本应用，例如，加一个食谱类，那么我们不得不在三个不同的地方做这些改变……”

访问者造访

访问者和一个导游携手合作。导游知道如何参观组合内的所有对象。导游引导访问者穿过组合，以便访问者边走边收集状态。一旦状态被收集，客户就可以让访问者在状态上执行各种操作。当需要新功能时，只要升级访问者。



访问者的优点

- 允许你添加操作到组合结构，而不改变结构本身。
- 添加新操作相对容易。
- 由访问者执行的操作的代码被集中化了。

访问者的用途和缺点

- 使用访问者模式时，组合类的封装被打破。
- 因为涉及到导游的功能，因此组合结构更加难以变化。

